

CHƯƠNG 4

QUẢN TRỊ MÃ NGUỒN DỰ ÁN VỚI GIT & CI/CD CƠ BẢN

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Hiểu được ý nghĩa việc quản lý mã nguồn dự án.
- Hiểu và sử dụng thành thạo Git cơ bản và GitHub workflow (branch, PR, merge).
- Giải thích được quy trình CI/CD và vai trò của GitHub Actions.
- Xây dựng pipeline CI/CD đơn giản để build và push Docker image.
- Quản lý phiên bản file cấu hình hệ thống theo tinh thần Infrastructure as Code.
- Làm việc nhóm hiệu quả qua Pull Request và code review.

4.1 Cơ bản về Git

4.1.1 Vai trò của quản lý mã nguồn dự án

Source code (mã nguồn) là tập hợp các lệnh được viết bằng một hoặc nhiều ngôn ngữ lập trình nhằm mô tả logic, quy trình xử lý và cách thức hoạt động của một chương trình. Đây là sản phẩm cốt lõi do lập trình viên tạo ra để xây dựng phần mềm, ứng dụng hoặc hệ thống.

Có thể hình dung source code giống như bản thiết kế chi tiết của một ngôi nhà: nó chứa đựng toàn bộ logic kinh doanh, quy tắc xử lý dữ liệu và cơ chế vận hành của hệ thống. Khi người dùng tương tác với ứng dụng, thực chất họ đang kích hoạt các lệnh đã được lập trình sẵn trong mã nguồn.

Source code được viết dưới dạng văn bản thuần, có thể đọc, hiểu và chỉnh sửa bởi con người. Tuy nhiên, máy tính không thể thực thi trực tiếp các dòng code này mà cần qua quá trình biên dịch (compilation) hoặc thông dịch (interpretation) để chuyển đổi thành ngôn ngữ máy (machine code).

Source code đóng vai trò trung tâm và quyết định trong toàn bộ vòng đời phát triển phần mềm. Mọi chức năng, giao diện và hành vi của ứng dụng đều được định nghĩa và kiểm soát bởi mã nguồn.

- **Nền tảng xây dựng phần mềm:** Source code là điểm khởi đầu của mọi sản phẩm công nghệ. Không có mã nguồn, lập trình viên không thể tạo ra bất kỳ chương trình nào, từ những ứng dụng đơn giản đến các hệ thống phức tạp.
- **Kiểm soát logic hệ thống:** Thông qua source code, lập trình viên định nghĩa cách xử lý dữ liệu, cách ứng dụng phản hồi với người dùng và cách các thành phần trong hệ thống tương tác với nhau, đảm bảo chương trình hoạt động đúng theo yêu cầu.

- **Hỗ trợ bảo trì và nâng cấp:** Khi cần sửa lỗi, tối ưu hiệu năng hoặc thêm tính năng mới, lập trình viên sẽ chỉnh sửa trực tiếp trên source code. Mã nguồn được tổ chức rõ ràng, có cấu trúc tốt sẽ giúp quá trình bảo trì trở nên hiệu quả và giảm thiểu rủi ro.
- **Hỗ trợ làm việc nhóm:** Trong các dự án lớn, nhiều lập trình viên cùng tham gia phát triển. Việc quản lý source code thông qua các hệ thống kiểm soát phiên bản (như Git) giúp các thành viên phối hợp nhịp nhàng, tránh xung đột code và đảm bảo tính nhất quán của dự án.

Các loại source code phổ biến:

Tùy theo mục đích và tầng kiến trúc của hệ thống, source code được phân loại thành nhiều loại khác nhau. Mỗi loại đảm nhận một vai trò cụ thể trong hệ thống.

Source code phía frontend

Frontend là phần mã nguồn chịu trách nhiệm xây dựng giao diện và trải nghiệm người dùng (User Interface - UI/UX). Đây là những yếu tố mà người dùng trực tiếp nhìn thấy và tương tác, bao gồm bố cục, màu sắc, phông chữ, nút bấm, hiệu ứng chuyển động...

Frontend thường được phát triển bằng các công nghệ như HTML, CSS và JavaScript (cùng với các framework/thư viện như React, Vue, Angular). Một frontend tốt không chỉ đẹp mắt mà còn phải đảm bảo tốc độ tải nhanh, khả năng tương tác mượt mà và thân thiện trên nhiều thiết bị (responsive).

Source code backend

Backend là phần “hậu phương” của ứng dụng, chịu trách nhiệm xử lý logic kinh doanh, thao tác dữ liệu, kết nối cơ sở dữ liệu và đảm bảo an ninh hệ thống. Đây là nơi xử lý các yêu cầu từ frontend, thực hiện các phép tính phức tạp và trả về kết quả phù hợp.

Backend thường được viết bằng các ngôn ngữ như Java, Python, Node.js, PHP, C#... cùng với các framework mạnh mẽ (Spring Boot, Django, Express, Laravel...). Backend mạnh mẽ giúp hệ thống hoạt động ổn định, xử lý được lượng lớn người dùng đồng thời và bảo vệ dữ liệu an toàn.

Source code mobile

Source code mobile dùng để phát triển các ứng dụng trên nền tảng di động. Tùy thuộc vào hệ điều hành, lập trình viên có thể sử dụng:

- Swift hoặc Objective-C cho iOS.
- Kotlin hoặc Java cho Android.

Hiện nay, các framework đa nền tảng như React Native hay Flutter cho phép viết một lần mã nguồn và triển khai trên cả iOS lẫn Android, giúp tiết kiệm đáng kể thời gian và chi phí phát triển. Source code mobile cần được tối ưu mạnh về hiệu năng và tiêu thụ tài nguyên do hạn chế về bộ nhớ, pin và sức mạnh xử lý của thiết bị di động.

Source code hệ thống

Đây là loại source code cấp thấp (low-level), được sử dụng để xây dựng hệ điều hành, driver thiết bị, compiler hoặc các phần mềm hệ thống cốt lõi. Thường được viết bằng C và C++ do khả năng kiểm soát trực tiếp tài nguyên phần cứng và hiệu suất cao.

Source code hệ thống đòi hỏi hiểu biết sâu sắc về kiến trúc máy tính, quản lý bộ nhớ và tối ưu hóa hiệu năng. Đây là nền tảng quan trọng nhất của toàn bộ hệ sinh thái

công nghệ, ảnh hưởng trực tiếp đến độ ổn định và hiệu suất của các lớp phần mềm cao hơn.

Quản lý mã nguồn (Source Code Management - SCM), hay còn gọi là hệ thống kiểm soát phiên bản (Version Control System - VCS), là quá trình tổ chức, theo dõi và quản lý sự thay đổi của source code trong suốt vòng đời của dự án phần mềm. Đây là một trong những yếu tố quan trọng nhất để đảm bảo tính chuyên nghiệp, hiệu quả và thành công của bất kỳ dự án phát triển phần mềm nào.

Trong môi trường làm việc thực tế, đặc biệt là các dự án có nhiều lập trình viên tham gia, việc quản lý mã nguồn tốt giúp giảm thiểu rủi ro, tăng năng suất làm việc và nâng cao chất lượng sản phẩm cuối cùng.

Vai trò chính của quản lý mã nguồn bao gồm:

- **Theo dõi toàn bộ lịch sử thay đổi:** Hệ thống quản lý mã nguồn ghi nhận mọi thay đổi của code (ai thay đổi gì, khi nào, và lý do). Lập trình viên có thể xem lại lịch sử, so sánh các phiên bản và khôi phục code về trạng thái trước đó nếu xảy ra lỗi.
- **Hỗ trợ làm việc nhóm hiệu quả:** Nhiều lập trình viên có thể cùng làm việc trên cùng một dự án mà không ghi đè lên code của nhau. Các tính năng như branching (tạo nhánh) và merging (hợp nhất) cho phép phát triển các tính năng song song, đồng thời giảm thiểu xung đột code (merge conflict).
- **Quản lý phiên bản và phát hành:** Cho phép tạo ra các phiên bản (version/release) ổn định của sản phẩm. Ví dụ: phiên bản 1.0, 1.1, 2.0... giúp đội ngũ dễ dàng quản lý cập nhật, vá lỗi và triển khai cho người dùng.
- **Tăng tính an toàn và khả năng khôi phục:** Khi gặp sự cố (mất code, xóa nhầm file, code bị hỏng), đội ngũ có thể khôi phục nhanh chóng về phiên bản trước. Điều này đặc biệt quan trọng trong các dự án lớn và dài hạn.
- **Hỗ trợ quy trình phát triển chuyên nghiệp:** Quản lý mã nguồn là nền tảng cho các quy trình hiện đại như CI/CD (Continuous Integration/Continuous Deployment), code review, pull request và tích hợp với các công cụ DevOps. Nó giúp đảm bảo chất lượng code trước khi đưa vào production.
- **Tăng khả năng bảo trì và mở rộng:** Với mã nguồn được quản lý tốt, việc bảo trì, nâng cấp tính năng mới hoặc chuyển giao dự án cho đội ngũ khác trở nên dễ dàng và ít rủi ro hơn.

Để quản lý mã nguồn hiệu quả, các lập trình viên thường sử dụng các công cụ phần mềm để làm việc đó một cách tự động.

4.1.2 Các giải pháp quản lý mã nguồn

Hiện nay có rất nhiều các giải pháp giúp quản lý mã nguồn được sử dụng phổ biến, trong đó có cả các giải pháp mã nguồn mở, cũng như cả bản trả phí.

a. GitHub (github.com)

Github là nền tảng lưu trữ mã nguồn lớn nhất thế giới, thuộc sở hữu của Microsoft. Đặc điểm nổi bật của giải pháp này đó là: Cộng đồng cực lớn (hơn 100 triệu lập trình viên), mạnh về Open Source, tích hợp tốt với GitHub Actions (CI/CD), Copilot (AI), và hệ sinh thái khổng lồ. Phù hợp cho cả cá nhân, dự án mở và doanh nghiệp.

Github có chức năng như một nguồn resource phát triển độc lập. Các developer có

thể xây dựng project, chia sẻ với cộng đồng và cộng đồng có thể sửa đổi. GitHub còn có chức năng như là một công cụ giúp một nhóm quản lý một repository cụ thể.

b. GitLab (gitlab.com)

Đây là nền tảng DevOps hoàn chỉnh, cung cấp tất cả công cụ trong một nền tảng duy nhất. GitLab dành cho những người quan tâm đến việc chạy repository của riêng mình và cũng là sự lựa chọn tốt để quản lý git repository trên một máy chủ trung tâm.

GitLab cho phép kiểm soát hoàn toàn source code repository. Đồng thời cũng cho người dùng quyền quyết định để chế độ public hay private cho từng source code repository.

Đặc điểm nổi bật: Hỗ trợ mạnh self-hosted (cài đặt trên server riêng), CI/CD tích hợp sẵn rất mạnh, bảo mật tốt, và mô hình open-core. Phù hợp cho đội ngũ muốn kiểm soát toàn bộ quy trình DevOps mà không phụ thuộc nhiều vào bên thứ ba.

c. Bitbucket (bitbucket.org)

BitBucket là một trong những kho lưu trữ lớn và là nhà của hàng nghìn dự án lớn nhỏ. Thuộc Atlassian (công ty làm Jira và Confluence). Đặc điểm nổi bật: Tích hợp sâu với Jira và các công cụ Atlassian, hỗ trợ Git và Mercurial, phù hợp với doanh nghiệp lớn đang dùng hệ sinh thái Atlassian. Giao diện chuyên nghiệp, tập trung vào đội ngũ phát triển.

d. Launchpad (launchpad.net)

Nền tảng do Canonical (công ty Ubuntu) phát triển. Launchpad có các công cụ để “biến” mã nguồn có thể cài đặt trên Ubuntu và các hệ thống Linux khác một cách dễ dàng. Launchpad hỗ trợ kiểm tra code, community translation và pull/push request và theo dõi vấn đề.

Đặc điểm nổi bật: Tích hợp chặt chẽ với hệ sinh thái Ubuntu/Linux, hỗ trợ Bazaar (dù cũng hỗ trợ Git), mạnh về bug tracking, bản dịch và xây dựng package. Thường được dùng cho các dự án mã nguồn mở liên quan đến Ubuntu.

e. Codebase (codebasehq.com)

Là một dịch vụ quản lý source code khác chuyên cho các doanh nghiệp. Codebase cung cấp tài khoản miễn phí nhưng lại hạn chế người dùng trong một dự án. Tuy nhiên, dự án đó sẽ có các công cụ quản lý code và quản lý dự án cực tốt. Nền tảng này chủ yếu tập trung vào đội ngũ chuyên nghiệp.

Đặc điểm nổi bật: Hỗ trợ nhiều hệ thống version control (Git, Mercurial, Subversion), kết hợp công cụ quản lý dự án (tickets, milestones). Phù hợp với đội ngũ nhỏ đến trung bình muốn giải pháp đơn giản, tích hợp project management.

f. Beanstalk (beanstalkapp.com)

Beanstalk cung cấp dịch vụ lưu trữ và quản lý source code đơn giản, hiệu quả hỗ trợ deployment cho server riêng và các môi trường web khác như AWS, DreamObjects, Rackspace, Heroku...

Nền tảng này tập trung vào việc triển khai code (deployment). Đặc điểm nổi bật: Giao diện đơn giản, mạnh về deploy code lên server, hỗ trợ Git và SVN. Phù hợp cho đội ngũ nhỏ, ưu tiên tốc độ deploy và quản lý phiên bản dễ dàng.

Dưới đây là bảng so sánh dựa trên các tiêu chí quan trọng đối với sinh viên và lập trình viên:

Bảng 4.1: So sánh các giải pháp quản lý mã nguồn

Tiêu chí	GitHub	GitLab	Bitbucket	Launchpad	Codebase	Beanstalk
Mức độ phổ biến	Rất cao (lớn nhất)	Cao	Trung bình - Cao	Trung bình (niche)	Thấp	Thấp
Mô hình hosting	Cloud (có Enterprise)	Cloud + Self-hosted mạnh	Cloud + Server	Cloud	Cloud	Cloud
CI/CD	GitHub Actions (mạnh)	Tích hợp sẵn rất mạnh	Pipelines	Trung bình	Cơ bản	Tốt về deploy
Tích hợp DevOps	Tốt	Xuất sắc (toàn diện nhất)	Tốt (với Jira)	Trung bình	Trung bình	Tập trung deploy
Open Source / Self-host	Không (proprietary)	Open-core, self-host miễn phí	Không	Có	Không	Không
Phù hợp cho	Open source, cá nhân, doanh nghiệp	Team DevOps, doanh nghiệp lớn	Team dùng Atlassian	Dự án Ubuntu/Linux	Team nhỏ	Team nhỏ cần deploy nhanh
Giá cả (Free tier)	Rất tốt	Tốt	Tốt	Miễn phí	Có gói trả phí	Có gói trả phí
Cộng đồng & Tài liệu	Xuất sắc	Rất tốt	Tốt	Trung bình	Trung bình	Thấp

Đối với sinh viên CNTT: Hãy bắt đầu với GitHub vì đây là nơi hầu hết nhà tuyển dụng và cộng đồng lập trình viên nhìn vào portfolio của ứng viên. Học GitLab nếu muốn hiểu sâu về DevOps và CI/CD. Và sử dụng Bitbucket nếu trường hoặc công ty đang dùng Jira.

4.1.3 Giới thiệu về Git và cài đặt

Git là hệ thống quản lý mã nguồn phân tán phổ biến nhất hiện nay, giúp theo dõi và lưu trữ lịch sử thay đổi của các tệp tin theo thời gian. Công cụ này cho phép lập trình viên làm việc nhóm hiệu quả, khôi phục trạng thái cũ và dễ dàng tích hợp mã nguồn.

Việc cài đặt Git rất nhanh chóng và đơn giản theo các bước sau:

- Tải bộ cài chính thức tại [Git SCM Downloads](#).
- Chọn phiên bản phù hợp cho Windows, macOS hoặc Linux.
- Chạy tệp vừa tải về, giữ nguyên các tùy chọn mặc định (nhấn Next liên tục) và hoàn tất việc cài đặt ứng dụng trên máy tính.

Những thiết lập cơ bản sau khi cài đặt Git:

Sau khi cài đặt xong, ta mở ứng dụng Git Bash (trên Windows) hoặc Terminal (trên macOS/Linux) và thiết lập thông tin cá nhân bằng các lệnh sau:

Cài đặt tên người dùng:

```
$git config --global user.name "Tên người dùng"
```

Cài đặt email:

```
$git config --global user.email "email@example.com"
```

Kiểm tra lại thông tin đã thiết lập:

```
$git config --global -l
```

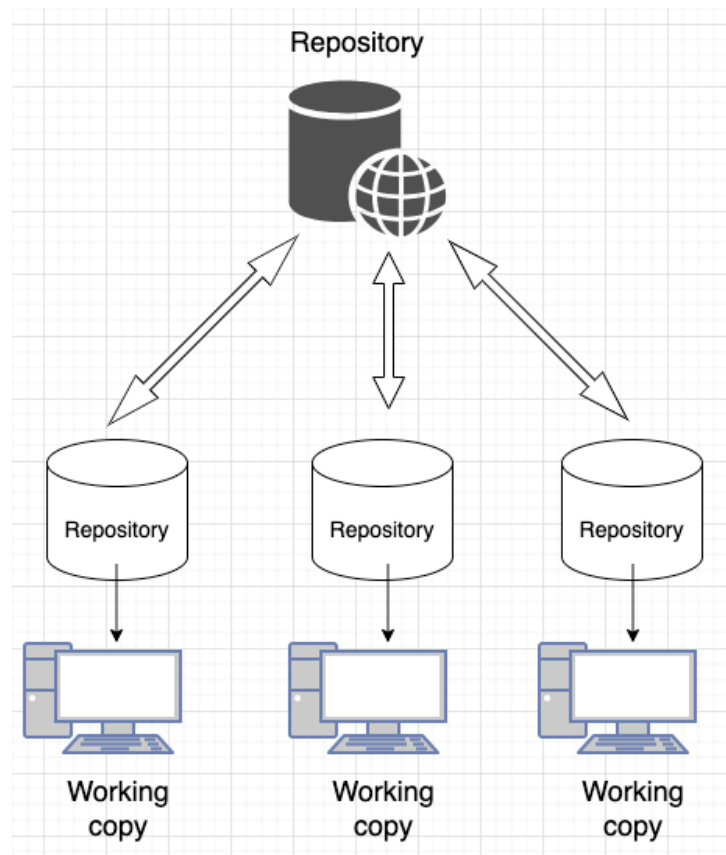
4.1.4 Các khái niệm cơ bản trong Git

a. Repository là gì?

Repository hay còn gọi là Repo, dịch ra tiếng Việt có nghĩa là kho, đây chính là nơi chứa tất cả mã nguồn cho một dự án được tạo bởi Git. Ta có thể hiểu một cách khác là Repository chính khai báo thư mục chứa dự án trên local hoặc remote. Một repo sẽ có hai cấu trúc dữ liệu chính đó là Object store và Index được lưu trữ ẩn trong thư mục `.git`.

Có hai loại repository là local repository và remote repository:

- **Local repository:** là repo được cài đặt trên máy tính của lập trình viên, repo này sẽ đồng bộ hóa với remote bằng các lệnh của Git.
- **Remote repository:** là repo được cài đặt trên server chuyên dụng, điển hình hiện nay là Github.



Hình 4.1: Mô hình local repository và remote repository

b. Branch là gì?

Đối với những dự án có nhiều thành viên tham gia thì mỗi thành viên sẽ nhận được nhiều task từ leader, vì vậy việc xử lý task này trên cùng một thời gian là rất khó vì dễ bị đụng code. Để giải quyết vấn đề này thì chúng ta sẽ sử dụng branch của Git, tương ứng với mỗi nhiệm vụ chúng ta sẽ tạo một branch và làm việc trên đó, các branch này sẽ hoạt động riêng lẻ và không ảnh hưởng lẫn nhau.

Vậy branch là những phân nhánh ghi lại luồng thay đổi của lịch sử, các hoạt động trên mỗi branch sẽ không ảnh hưởng lên các branch khác nên có thể tiến hành nhiều thay đổi đồng thời trên một repository giúp giải quyết nhiều nhiệm vụ cùng lúc.

Khi chúng ta tạo một repository thì Git sẽ thiết lập branch mặc định là master, nghĩa là nó sẽ tự tạo một branch master và mọi hoạt động của bạn lúc này đều nằm trên branch master.

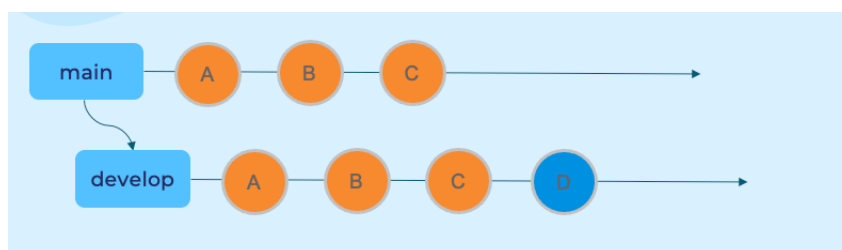
c. Commit

Khi cần lưu lại các thay đổi vào Repository, người dùng cần thực hiện việc tạo ra một commit. Mỗi commit trong Git là đại diện cho một phiên bản của mã nguồn, nó bao gồm các thông tin sau:

- Danh sách các thay đổi trên các tập tin, gọi là “snapshot”.
- Thông tin của người tạo ra commit và thời gian tạo.
- Mô tả của commit.

Ngoài ra, mỗi commit sẽ được gắn liền với một mã băm duy nhất dài 40 ký tự gọi là SHA-1. Đây được xem như là mã định danh của commit đó và sẽ không có hai SHA-1 nào trùng nhau trong một mã nguồn.

Ví dụ, một repository có nhánh “main” với ba commit là A, B, C. Người dùng tiến hành tách một nhánh mới là “develop”. Nhánh này cũng sẽ có ba commit ban đầu là A, B, C. Tiếp theo người dùng chuyển sang làm việc trên nhánh develop, và thực hiện tạo ra commit D. Thì lúc này, nhánh develop sẽ có bốn commit, tuy nhiên nhánh “main” vẫn sẽ giữ nguyên là ba commit A, B, C.



Hình 4.2: Ví dụ lịch sử commit trên hai nhánh main và develop

Các thuật ngữ quan trọng khác khi sử dụng với Git như:

Bảng 4.2: Các thuật ngữ quan trọng trong Git

Thuật ngữ	Ý nghĩa
Branch	Mỗi nhánh lưu trữ một dòng lịch sử commit riêng trong kho lưu trữ, có thể tách thành nhiều nhánh và hợp nhất nhánh
Checkout	Lệnh dùng để chuyển đổi giữa các nhánh hoặc các phiên bản cụ thể có trong repository

Thuật ngữ	Ý nghĩa
HEAD	Đại diện cho commit mới nhất trong nhánh làm việc hiện tại. Khi có một commit mới được tạo ra, HEAD sẽ trở thành commit đó.
Merge	Kết hợp các thay đổi từ nhánh này vào một nhánh khác bằng cách hợp nhất lịch sử commit, thường sẽ tạo ra một commit mới với nội dung kết hợp trong nhánh được merge
Rebase	Khác với Merge, tuy nhiên Rebase sẽ sắp xếp lại các commit từ hai nhánh theo hướng tuyến tính để hợp nhất 2 nhánh, và không tạo ra commit Merge
Conflict	Là xung đột giữa các thay đổi trên cùng một tập tin khi tiến hành hợp nhất các nhánh
Origin	Tên mặc định được Git sử dụng để chỉ remote repository đã kết nối với repository cục bộ. Nó chứa thông tin địa chỉ của kho lưu trữ từ xa
Clone	Là quá trình sao chép toàn bộ nội dung repository từ kho lưu trữ từ xa về máy cục bộ
Push	Cập nhật các commit mới từ kho lưu trữ cục bộ đến kho lưu trữ từ xa
Fetch	Lấy các commit thay đổi từ kho lưu trữ từ xa nhưng không tiến hành hợp nhất với nhánh đang làm việc
Pull	Là sự kết hợp giữa fetch và merge đối với nhánh đang làm việc
Tag	Đánh dấu một phiên bản cụ thể của dự án, có thể sử dụng Checkout để chuyển đến phiên bản này. Thường được sử dụng để phát hành các phiên bản chính thức của phần mềm
Git config	Thiết lập các cấu hình cho Git như thông tin người dùng, trình soạn thảo mặc định,...
Cherry-pick	Sao chép một commit từ nhánh khác mà không cần phải gộp nhánh
Fork	Tạo ra một bản sao repository độc lập từ một nguồn khác
.gitignore	Dùng để liệt kê các file muốn không Git theo dõi và Git sẽ bỏ qua mọi thay đổi trong các file đó
Git flow	Là quy trình làm việc (workflow) trong Git, được thiết kế để giúp quản lý các dự án có vòng đời phát triển dài và phức tạp.

4.2 Quy trình CI/CD cơ bản

4.2.1 Thế nào là CI/CD

CI/CD là viết tắt của Continuous Integration (tích hợp liên tục) và Continuous Delivery/Deployment (chuyển giao hoặc triển khai liên tục). Đây là một quy trình tự động hóa giúp rút ngắn khoảng cách giữa việc phát triển và phát hành phần mềm. Thay vì triển khai thủ công, CI/CD cho phép tự động build, test và deploy mỗi khi lập trình viên cập nhật mã nguồn. Mô hình này đang ngày càng phổ biến trong các nhóm phát triển phần mềm hiện đại, đặc biệt khi kết hợp cùng DevOps và Agile, nhằm đảm bảo sản phẩm được cải tiến liên tục và nhanh chóng đến tay người dùng.

Ưu điểm khi sử dụng CI/CD:

- Tự động hóa quy trình kiểm thử giúp phát hiện lỗi sớm và giảm rủi ro khi triển khai.
- Giúp các thành viên trong nhóm nắm bắt thông tin kịp thời, tăng tính phối hợp.
- Tiết kiệm thời gian và công sức nhờ loại bỏ thao tác kiểm thử thủ công.
- Phản hồi và xử lý sự cố nhanh chóng, hỗ trợ phát triển phần mềm linh hoạt.

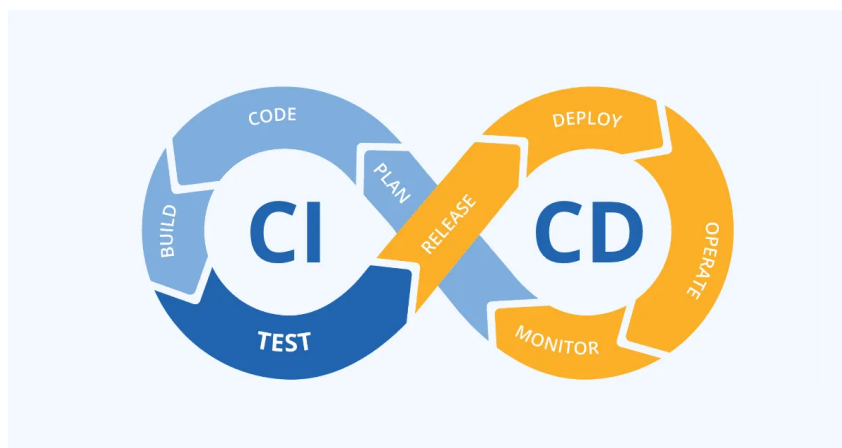
Tuy nhiên trong quá trình sử dụng CI/CD cũng cần lưu ý những vấn đề sau có thể

ảnh hưởng đến hiệu suất hoạt động của hệ thống và khả năng áp dụng:

- Yêu cầu người dùng có kiến thức vững và kinh nghiệm để vận hành hệ thống hiệu quả.
- Hệ thống CI/CD khá phức tạp, cần cấu hình và bảo trì thường xuyên.
- Chi phí đầu tư ban đầu cao, nhất là với doanh nghiệp nhỏ.
- Một số dự án cũ hoặc hệ thống đặc thù có thể không tương thích với CI/CD.

CI/CD nên được áp dụng càng sớm càng tốt, kể cả với dự án nhỏ hay cá nhân. Việc triển khai sớm giúp đảm bảo chất lượng mã nguồn, giảm thiểu lỗi khi release và đẩy nhanh tốc độ phát hành phần mềm. Đặc biệt, trong các dự án yêu cầu cập nhật thường xuyên hoặc triển khai nhanh như sản phẩm MVP, CI/CD giúp nhóm phát triển tiết kiệm thời gian và tối ưu quy trình.

Tuy nhiên, nếu đội ngũ chưa quen với khái niệm test tự động, chưa đủ kỹ năng quản lý pipeline hay thiếu nền tảng hạ tầng phù hợp thì việc triển khai có thể gây gián đoạn. Ở những tình huống này, nên đầu tư thời gian học tập hoặc áp dụng CI/CD từng phần thay vì toàn bộ.



Hình 4.3: Quy trình hoạt động CI/CD

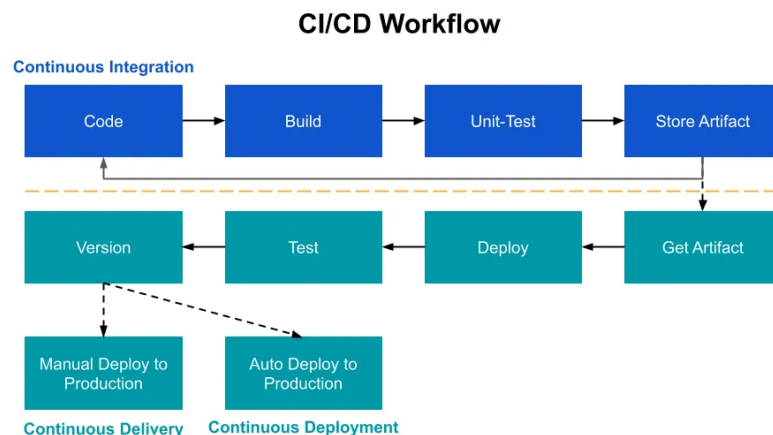
Ngoài yếu tố con người, hạ tầng cũng là điều cần lưu ý. Chẳng hạn, nếu chúng ta đang triển khai CI/CD trên máy chủ ảo, việc chọn dịch vụ lưu trữ có tốc độ đọc/ghi cao sẽ giúp giảm thời gian build và deploy đáng kể – đây là yếu tố quan trọng khi tối ưu hiệu suất cho quy trình tự động hóa.

4.2.2 Quy trình CI/CD trong devops

Một quy trình làm việc theo mô hình CI/CD thường bao gồm hai giai đoạn chính: phát triển (Development) và triển khai (Deployment).

Ở giai đoạn phát triển, lập trình viên sẽ tạo nhánh (branch) mới để phát triển tính năng. Khi hoàn tất, họ gửi pull request để hợp nhất mã vào nhánh chính. Lúc này, pipeline CI/CD tự động được kích hoạt để thực hiện các bước như build, kiểm thử, kiểm tra lint và báo cáo lỗi nếu có.

Khi pull request được duyệt và hợp nhất, hệ thống chuyển sang giai đoạn triển khai. Mã nguồn sẽ được build thành sản phẩm và tự động đẩy lên các nền tảng phân phối như TestFlight (iOS) hoặc Firebase App Distribution (Android). Kết quả được thông báo qua các kênh như Slack để nhóm kiểm thử tiếp tục đánh giá.



Hình 4.4: CI/CD Workflow

Khó khăn khi triển khai quy trình CI/CD:

- **Chọn công cụ phù hợp:** Có quá nhiều nền tảng CI/CD (như CircleCI, Bitrise, GitHub Actions...), mỗi công cụ có điểm mạnh riêng, khiến việc lựa chọn cần thời gian thử nghiệm và đánh giá kỹ.
- **Tốn thời gian cấu hình ban đầu:** Việc thiết lập pipeline, tích hợp test và build tự động cần đầu tư thời gian và kiến thức, đặc biệt nếu team chưa có kinh nghiệm trước đó.
- **Thiếu kỹ năng về unit test và viết code sạch:** CI/CD yêu cầu code dễ kiểm thử. Những team chưa quen với test tự động sẽ gặp khó khăn trong việc viết unit test chuẩn.
- **Sự kháng cự từ thành viên trong team:** Một số lập trình viên có thể thấy quy trình mới làm việc gò bó, mất thời gian, hoặc không quen với việc kiểm soát chặt chẽ qua pipeline.
- **Chi phí công cụ hoặc tài nguyên hạn chế:** Một số công cụ CI/CD mạnh có giới hạn về tài nguyên miễn phí, hoặc cần đầu tư thêm về hạ tầng (ví dụ VPS mạnh để build nhanh).
- **Thiếu sự đồng thuận trong team:** Việc áp dụng CI/CD hiệu quả cần sự phối hợp và đồng lòng từ cả nhóm. Nếu không có sự thống nhất, quy trình dễ bị đứt gãy hoặc nửa vời.
- **Cần liên tục cập nhật và cải tiến quy trình:** CI/CD không phải là “cài một lần là xong”. Team phải thường xuyên tinh chỉnh để phù hợp với tốc độ và quy mô dự án.

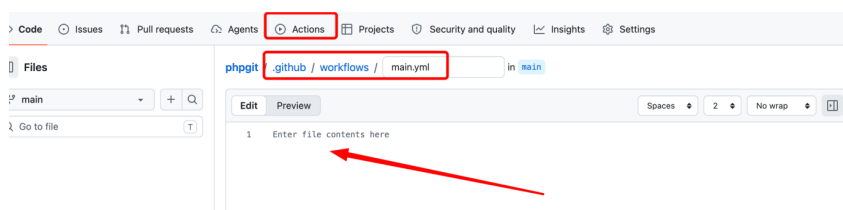
4.2.3 Vai trò và ứng dụng Github Actions

GitHub Actions là một nền tảng CI/CD tích hợp sẵn trên GitHub, cho phép người dùng có thể tự động hóa toàn bộ quy trình phát triển phần mềm từ xây dựng, kiểm thử, đến triển khai. Với GitHub Actions, ta có thể tạo các workflow (quy trình làm việc) để tự động thực hiện các tác vụ khi có các sự kiện cụ thể xảy ra trong repository của dự án. Hệ thống GitHub Actions sẽ giúp ta thiết lập các tác vụ CI/CD trên repository GitHub, người dùng chỉ có thể dùng được nó khi dùng Github và nó đã và đang trở thành công cụ CI/CD sử dụng nhiều nhất hiện tại.

Những thành phần của GitHub Actions

Workflow

GitHub Workflow là một tập hợp các quy trình tự động hóa được định nghĩa trong repository GitHub, giúp quản lý và thực hiện các công việc như build, test, và deploy phần mềm. Workflows được cấu hình thông qua các tệp YAML và lưu trữ trong thư mục `.github/workflows` của repository.



Hình 4.5: Thiết lập workflow trong github

Jobs

Jobs là các tập hợp các bước (steps) cần thực hiện trong workflow. Mỗi job chạy trên một runner riêng biệt, có thể chạy song song hoặc theo thứ tự phụ thuộc lẫn nhau.

- **Runner:** Là môi trường (ví dụ: Ubuntu, Windows, macOS) nơi các job được thực thi.
- **Isolation:** Mỗi job chạy trong môi trường cô lập để đảm bảo sự độc lập và tránh xung đột.

```
1 # Sample workflow for building and deploying a Nuxt site to GitHub Pages
2 #
3 # To get started with Nuxt see: https://nuxtjs.org/docs/get-started/installation
4 #
5 name: Deploy Nuxt site to Pages
6
7 on:
8   # Runs on pushes targeting the default branch
9   push:
10     branches: ["main"]
11
12   # Allows you to run this workflow manually from the Actions tab
13   workflow_dispatch:
14
15 # Sets permissions of the GITHUB_TOKEN to allow deployment to GitHub Pages
16 permissions:
17   contents: read
18   pages: write
19   id-token: write
20
21 # Allow only one concurrent deployment, skipping runs queued between the run in-progress and latest queued.
22 # However, do NOT cancel in-progress runs as we want to allow these production deployments to complete.
23 concurrency:
24   group: "pages"
25   cancel-in-progress: false
26
27 jobs:
28   # Build job
29   build:
30     runs-on: ubuntu-latest
31     steps:
32       - name: Checkout
33         uses: actions/checkout@v4
```

Hình 4.6: File cấu hình của workflow cho dự án Nextjs

Ta có thể thấy các thông tin cấu hình với các thẻ: name, permissions, jobs, build... như ở trong tập tin cấu hình trên.

Runners

Runners là các máy chủ hoặc máy ảo nơi các jobs trong workflow được thực thi. GitHub cung cấp runners mặc định (hosted runners) và ta cũng có thể tự thiết lập runners của riêng mình (self-hosted runners).

- **GitHub-hosted runners:** GitHub cung cấp các runners được quản lý và bảo trì bởi GitHub. Các runners này hỗ trợ nhiều hệ điều hành và cấu hình khác nhau.
- **Self-hosted runners:** Ta có thể thiết lập và quản lý runners của riêng mình trên các máy chủ vật lý hoặc máy ảo mà mình kiểm soát. Điều này cho phép tùy chỉnh môi trường chạy theo nhu cầu cụ thể của dự án.

```
1 jobs:
2   build:
3     runs-on: ubuntu-latest
```

Hình 4.7: Thiết lập môi trường để chạy ứng dụng

Actions

Actions là các nhiệm vụ (tasks) được thực thi trong một workflow. Một action có thể là một lệnh shell hoặc một phần mềm được đóng gói sẵn giúp thực hiện các nhiệm vụ phổ biến.

- **Marketplace:** GitHub Actions có một marketplace nơi ta có thể tìm thấy nhiều actions do cộng đồng và GitHub cung cấp.
- **uses:** Từ khóa để sử dụng một action trong bước của job.

```
1 steps:
2   - name: Checkout code
3     uses: actions/checkout@v2
4
5   - name: Set up Node.js
6     uses: actions/setup-node@v2
7     with:
8       node-version: '14'
```

Hình 4.8: Thiết lập các nhiệm vụ cụ thể sẽ thực thi

Events

Events là các sự kiện kích hoạt workflows. GitHub cung cấp nhiều sự kiện khác nhau, chẳng hạn như khi có push, pull request, hoặc issue mới.

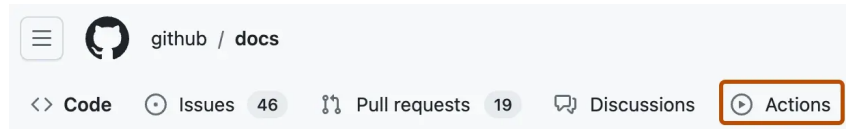
- **on:** Định nghĩa sự kiện kích hoạt workflow.
- Các sự kiện phổ biến: `push`, `pull_request`, `schedule`, `workflow_dispatch`

```
1 on:
2   push:
3     branches:
4       - main
5   pull_request:
6     branches:
7       - main
```

Hình 4.9: Thiết lập các sự kiện

Ví dụ thiết lập Workflow trong Github Actions: Dưới đây là một ví dụ về workflow trên trang chủ GitHub, cài thư viện bats trong môi trường ubuntu và kiểm tra version mới cài đặt:

Trước tiên cần truy xuất GitHub Actions ở 1 repository bất kì bằng cách nhấn vào menu Actions như hình bên dưới:



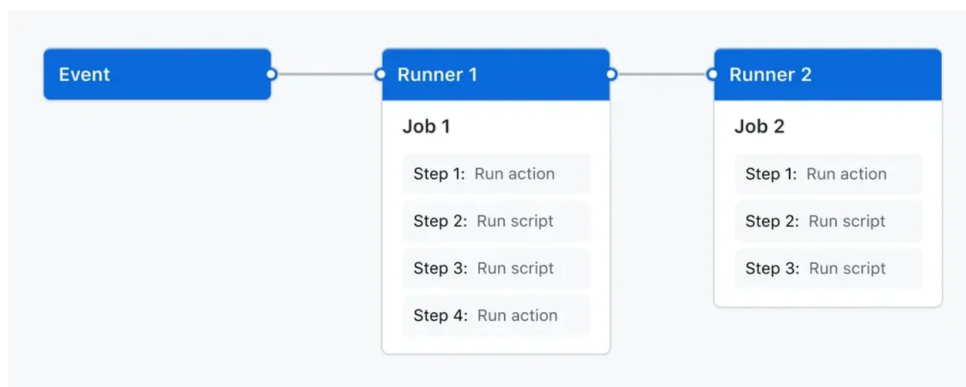
Hình 4.10: Menu Actions trên GitHub

Các bước thực hiện tiếp theo:

- Tạo thư mục `.github/workflows/` trong repository.
- Trong thư mục `.github/workflows/`, tạo một file `learn-github-actions.yml` và thêm vào đoạn code cấu hình bên dưới.

```
name: learn-github-actions
run-name: ${github.actor} is learning GitHub Actions
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install -g bats
      - run: bats -v
```

Biểu đồ mô tả workflow như sau:



Hình 4.11: Biểu đồ mô tả workflow GitHub Actions

Giải thích các thông số:

- **name** (không bắt buộc): Tên của workflow sẽ xuất hiện trong tab "Actions" của repository GitHub. Nếu trường này bị bỏ qua, tên của file workflow sẽ được dùng để thay thế.

- **run-name:** Tên động, có thể thay đổi qua các lần chạy, ví dụ như tên người đã trigger workflow chạy.
- **on:** Định nghĩa trigger mà workflow này sẽ chạy, ví dụ push event.
- **jobs:** Danh sách các công việc trong workflow.
- **check-bats-version:** tên job.
- **runs-on:** môi trường thực thi (ubuntu, windows server, ...).
- **steps:** các bước trong 1 job.
- **uses:** hành động xảy ra trong từng bước.

4.3 Thực hành quản lý mã nguồn với Git

4.3.1 Cấu hình Git trên local

4.3.2 Thiết lập và thực hành các thao tác cơ bản

Các lệnh Git cơ bản để Cấu hình và khởi tạo Repository

Git init

Git init dùng để khởi tạo một repository mới tại thư mục hiện tại, chỉ cần chạy một lần duy nhất lúc tạo mới repository.

```
$git init
```

Git clone

Git clone tạo ra bản sao làm việc cục bộ của một repository ở kho lưu trữ từ xa thông qua URL, chỉ cần chạy một lần duy nhất khi chưa có bản sao ở máy cục bộ.

Cú pháp:

```
$git clone [url]
```

Git config

Git config có chức năng dùng để thiết lập các thông số như thông tin người dùng, hành vi của Git trong phạm vi một repository hoặc toàn bộ repository trong máy cá nhân. Git config thường được sử dụng ngay sau khi cài đặt Git, hoặc trong trường hợp làm việc ở nhiều repository với nhiều tên tác giả khác nhau.

Cú pháp:

```
$git config [--global] <thuộc tính cấu hình> <giá trị>
```

Trong đó, cờ `--global` được sử dụng để áp dụng các thiết lập vào tất cả các repository trên máy tính. Nếu không có cờ `--global`, các thiết lập sẽ chỉ áp dụng cho repository hiện tại đang được thao tác.

Ví dụ:

```
$git config --global user.email "example@abc.com"
$git config --global user.name "Admin IT"
```

Xem các cấu hình hiện tại:

```
$git config --global --list
```

Thiết lập editor mặc định cho Git:

```
$git config --global core.editor "code --wait"
```

4.3.3 Làm việc với các lệnh Git cơ bản để Quản lý file và thư mục

a. Lệnh Git status

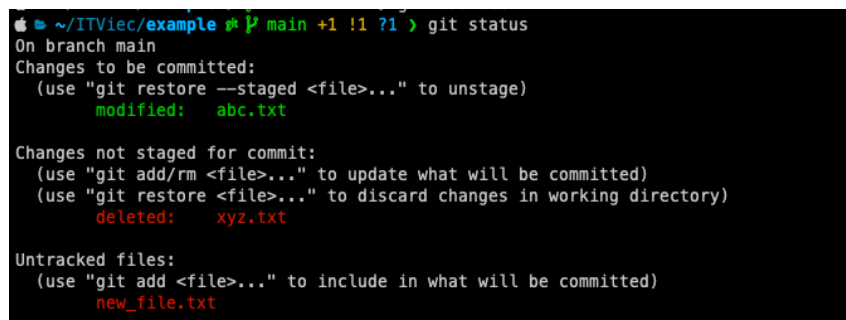
Git status có chức năng hiển thị trạng thái hiện tại của các tập tin trong khu vực làm việc và khu vực Staging.

Cú pháp:

```
$git status
```

Với Git status, chúng ta có thể thấy được:

- Những tập tin có sự thay đổi đã được thêm vào Staging Area để chuẩn bị commit.
- Những tập tin có thay đổi hoặc bị xóa nhưng chưa được thêm vào Staging Area.
- Các tập tin mới được tạo nhưng chưa được lưu trữ trong repository (untracked).
- Trạng thái của branch hiện tại và các commit đã thực hiện.



```
🍏 ~/ITViec/example 🐚 main +1 !1 ?1 > git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   abc.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    xyz.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        new_file.txt
```

Hình 4.12: Xem trạng thái với git status

b. Lệnh Git add

Trước khi thực hiện commit các thay đổi vào Repository, các thay đổi đó cần được thêm vào khu vực Staging, lúc đó chúng ta sẽ cần đến lệnh Git add.

Cú pháp:

```
$git add <tập tin hoặc thư mục>
```

Một số trường hợp sử dụng lệnh Git add như sau:

- Thêm toàn bộ thư mục: `$git add folder_name/`
- Thêm tập tin cụ thể: `$git add file_name`

- Thêm tất cả các tập tin có sự thay đổi vào khu vực Staging: `$git add .`

c. Git rm

Lệnh git rm được sử dụng để xóa file khỏi khu vực làm việc hoặc khu vực Staging. Sau khi thực hiện, file bị xóa sẽ không còn được theo dõi trong repository. Tuy nhiên điều này chỉ hoàn tất khi thực hiện commit thay đổi đó.

Cú pháp:

```
$git rm <tên thư mục hoặc tên file>
```

Một số lệnh Git cơ bản để làm việc với Branch

Git branch

Git branch dùng để quản lý nhánh trong repository. Branch là đơn vị để tách biệt công việc đang thực hiện, điều này giúp việc phát triển các tính năng mới không làm ảnh hưởng đến branch chính đang hoạt động ổn định.

Cú pháp:

```
$git branch <tùy chọn> <tên branch>
```

Với việc kết hợp các tùy chọn, git branch được sử dụng trong các trường hợp như:

Tạo branch mới:

```
$git branch <tên branch mới>
```

Xóa branch:

```
$git branch -d <tên branch>
```

Nếu branch chưa được hợp nhất vào branch khác, ta cần sử dụng `-D`:

```
$git branch -D <tên branch>
```

Đổi tên branch hiện tại:

```
$git branch -m <tên mới>
```

Kiểm tra danh sách branch đang có:

```
$git branch <-a | -r>
```

Trong đó, `-a` để hiển thị tất cả các branch (ở cục bộ và kho từ xa), `-r` để hiển thị các branch trên kho lưu trữ từ xa, nếu không có tùy chọn đi kèm, git branch sẽ hiển thị các branch ở cục bộ.

Git checkout

Git checkout được sử dụng để chuyển đổi trạng thái làm việc giữa các branches hoặc các phiên bản.

Cú pháp:

```
$git checkout <tên branch>
```

Ngoài ra, ta có thể thực hiện một số chức năng khác khi kết hợp với các tùy chọn:
Sử dụng `-b` để thực hiện đồng thời việc tạo branch mới và chuyển trạng thái làm việc sang branch mới tạo:

```
$git checkout -b <tên branch mới>
```

Sử dụng `--` để khôi phục trạng thái trước đó của tập tin:

```
$git checkout -- <tên tập tin>
```

Chuyển đến trạng thái làm việc của một commit:

```
$git checkout <commit id>
```

Git switch

Đây là lệnh có chức năng chuyển đổi trạng thái làm việc giữa các branches. Điều này giúp tách biệt chức năng với lệnh git checkout (có nhiều tác vụ khác như chuyển commit, khôi phục tập tin).

Cú pháp:

```
$git switch <tên branch>
```

Lệnh git switch cũng có tùy chọn để thực hiện đồng thời công việc tạo branch mới và chuyển sang branch đó:

```
$git switch -c <tên branch>
```

Các lệnh Git cơ bản để Stage và Commit

Git stash

Lệnh này dùng để lưu trữ tạm thời các thay đổi đã được thực hiện nhưng chưa được commit và khiến cho khu vực làm việc trở nên sạch sẽ.

Cú pháp:

```
$git stash
```

Ví dụ:

Khu vực làm việc đang có một số tập tin thay đổi:

Khu vực làm việc sau khi sử dụng git stash:

Các cách sử dụng Git stash

Trường hợp nếu muốn khôi phục các thay đổi đã lưu trước đó, ta có thể sử dụng các cách sau:

Khôi phục một stash cụ thể và vẫn lưu trữ lại stash đó:

```
➤ ~/ITViec/example ✱ P main !2 > git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   abc.txt
        modified:   xyz.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Hình 4.13: Khu vực làm việc trước khi sử dụng git stash

```
➤ ~/ITViec/example ✱ P main !2 > git stash
Saved working directory and index state WIP on main: c0f92f5 init
➤ ~/ITViec/example ✱ P main *1 > git status
On branch main
nothing to commit, working tree clean
```

Hình 4.14: Khu vực làm việc sau khi sử dụng git stash

```
$git stash apply <stash id>
```

Khôi phục stash mới nhất và xóa bỏ stash đó:

```
$git stash pop
```

Kiểm tra danh sách stash hiện tại:

```
$git stash list
```

Git commit

Lệnh này sẽ tạo ra các commit lưu lại các thay đổi được xảy ra trên tập tin vào repository. Mỗi commit sẽ có một id định danh duy nhất được sử dụng để thực hiện các thao tác và quản lý. Mỗi lần commit, nên cần đính kèm một nội dung mô tả tổng quát những thay đổi được thực hiện trong commit.

Cú pháp:

```
git commit -m "nội dung mô tả"
```

Mỗi commit sẽ chứa các thông tin về:

- Các thay đổi đã thực hiện
- Tác giả của commit
- Thời gian commit
- Nội dung mô tả các thay đổi

Trường hợp cần thay đổi nội dung của một commit trước đó, ta có thể thực hiện bằng cú pháp:

```
$git commit --amend
```

Bên cạnh đó, để kết hợp thao tác git add và git commit, ta có thể thực hiện như sau:

```
$git commit -a -m "nội dung mô tả"
```

Các lệnh Git cơ bản để Chia sẻ Repository và làm việc nhóm

Git remote

Git remote dùng để kết nối kho lưu trữ cục bộ với kho lưu trữ từ xa. Kho lưu trữ từ xa có thể được đặt tên riêng (thường sử dụng origin) để tránh phải nhớ URL mỗi khi thực hiện các thao tác có liên quan.

Cú pháp:

```
$git remote <command> <tên bí danh> <remote URL>
```

Một số trường hợp sử dụng:

Thêm một kho lưu trữ từ xa mới:

```
$git remote add origin <remote url>
```

Xóa kết nối một kho lưu trữ từ xa:

```
$git remote remove origin
```

Đổi tên bí danh của kho lưu trữ từ xa:

```
$git remote rename origin new_name
```

Hiển thị danh sách các kho lưu trữ từ xa hiện có:

Git push

Git push có chức năng gửi các commit hoặc tags ở kho lưu trữ cục bộ đến kho lưu trữ từ xa. Git push cần xác định 2 tham số là tên kho lưu trữ từ xa và tên branch đang thực hiện

Cú pháp:

```
$git push origin <tên branch>
```

Nếu trường hợp đây là lần đầu tiên đẩy branch lên kho lưu trữ từ xa, ta có thể sử dụng tùy chọn -u (upstream), điều này sẽ thiết lập branch hiện tại theo dõi branch tương ứng trên kho lưu trữ từ xa. Những lần thực hiện git push sau đó, ta không cần phải chỉ định tên branch.

```
$git push -u origin <tên branch>
```

Git fetch

Lệnh git fetch được sử dụng để cập nhật tất cả các thay đổi mới từ kho lưu trữ từ xa về kho lưu trữ cục bộ mà không tự động hợp nhất chúng vào nhánh cục bộ. Đây là một lệnh an toàn, cho phép kiểm tra các thay đổi mới trước khi quyết định hợp nhất chúng.

Cú pháp:

```
$git fetch <tên remote>
```

Ngoài ra, để lấy tất cả branches, tags và commits từ kho lưu trữ từ xa, có thể sử dụng tùy chọn `--all`

```
$git fetch --all
```

Git merge

Git merge được sử dụng để hợp nhất các thay đổi từ branch này sang branch khác. Khi hợp nhất, nó sẽ áp dụng các commit của branch được merge vào branch hiện tại. Trường hợp giữa hai branch có một số commit mới kể từ lần tách branch trước đó, lệnh merge sẽ tạo ra một commit hợp nhất ghi lại nội dung kết hợp giữa hai branch.

Cú pháp:

```
$git merge <tên branch>
```

Git rebase

Git rebase có chức năng tương tự như Git merge, dùng để hợp nhất các thay đổi từ branch này sang branch hiện tại. Tuy nhiên, khác với git merge, git rebase sẽ lấy tất cả commit từ branch được merge chèn lên đầu trong danh sách commit của branch hiện tại và không tạo ra commit hợp nhất.

Cú pháp:

```
$git rebase <tên branch>
```

Git pull

Git pull sẽ lấy về tất cả thay đổi từ kho lưu trữ từ xa và hợp nhất chúng vào branch hiện tại, hay nói cách khác, git pull chính là sự kết hợp giữa git fetch và git merge.

Cú pháp:

```
$git pull <tên remote> <tên branch>
```

Tuy nhiên, ta có thể thay thế việc sử dụng git merge bằng git rebase trong lệnh git pull bằng cách sử dụng tùy chọn `--rebase` (hoặc viết tắt là `-r`):

```
$git pull -r <tên remote> <tên branch>
```

Các lệnh Git cơ bản để Kiểm tra thay đổi và lịch sử:

Git log

Lệnh dùng để hiển thị toàn bộ lịch sử commit theo thứ tự thời gian của một repository. Điều này giúp cung cấp bối cảnh và lịch sử của một repository.

Cú pháp:

```
$git log
```

Git diff

Lệnh git diff được dùng để so sánh sự khác biệt giữa các phiên bản khác nhau của repository.

Cú pháp:

```
$git diff <tùy chọn>
```

Trường hợp sử dụng Git diff

So sánh khác biệt giữa 2 commit

```
$git diff <commit id 1> <commit id 2>
```

So sánh khác biệt giữa các tập tin trong khu vực Staging và phiên bản đã được commit trước đó.

```
$git diff --staged
```

So sánh khác biệt giữa hai branches

```
$git diff <branch 1> <branch 2>
```

Bài thực hành

Bài 1. Thiết lập kết nối giữa tài khoản Github và máy tính cá nhân

Yêu cầu: Tạo kết nối để đồng bộ dữ liệu giữa tài khoản Github và máy tính cá nhân.

Bài 2. Thực hành các thao tác cơ bản với Git

- Tạo 1 thư mục bất kỳ trên máy tính, khởi tạo thư mục với git init, sau đó cấu hình remote đến địa chỉ repository trên tài khoản github.
- Thực hiện thay đổi nội dung trong thư mục để thực hành các lệnh cơ bản như: git add, git commit, git status, git log, git push...

Bài 3. Thực hành clone dữ liệu từ Github

- Sử dụng git clone để đồng bộ dữ liệu từ repo trên Github về máy tính cá nhân.
- Thực hiện các thao tác thay đổi dữ liệu trong thư mục và commit lên github.